

프로그래밍 입문

깃허브 정리

2022864047 채수형

<https://github.com/Akaaaa12345/Python>

프로그래밍과 파이썬소개

Commits on Mar 12, 2026

Add files via upload Akaaaa12345 authored on Mar 12	Verified9b97cb4
Add files via upload Akaaaa12345 authored on Mar 12	Verified56855c2
Delete 0306/hello.py Akaaaa12345 authored on Mar 12	Verified33760bb
Add files via upload Akaaaa12345 authored on Mar 12	Verifiedc609aad

1. 프로그래밍과 파이썬 소개

 print() 함수 핵심 정리

print() 는 파이썬에서 가장 기본적이고 자주 쓰이는 화면 출력 함수입니다. 컴퓨터에게 "이 내용을 모니터 화면에 보여줘"라고 명령하는 역할을 합니다.

1. 문자열(Text) 출력 규칙

- 따옴표 필수: 글자나 문장을 출력할 때는 반드시 양옆을 ****큰따옴표(" ")**나 **작은따옴표(' ')**로 감싸야 합니다.
- 에러 주의: 따옴표를 생략하거나 한쪽만 쓰면 **SyntaxError** (문법 오류)가 발생합니다.

```
print("안녕하세요")           # 올바른 예시
print('파이썬은 강력합니다.') # 올바른 예시
# print(안녕하세요)          # 에러 발생 (따옴표 누락)
```

2주차

변수와 수식

Commits on Mar 27, 2026			
Add files via upload	Verified	128c47d	📄 📁 ↔
Akaaaa12345 authored on Mar 27			
Commits on Mar 20, 2026			
Add files via upload	Verified	be1ec46	📄 📁 ↔
Akaaaa12345 authored on Mar 20			
Add files via upload	Verified	437a450	📄 📁 ↔
Akaaaa12345 authored on Mar 20			
Add files via upload	Verified	7df9148	📄 📁 ↔
Akaaaa12345 authored on Mar 20			
Create readme.md	Verified	8fd0130	📄 📁 ↔
Akaaaa12345 authored on Mar 20			
Delete 0320	Verified	ea2eebd	📄 📁 ↔
Akaaaa12345 authored on Mar 20			
readme.md	Verified	c02d4c4	📄 📁 ↔
Akaaaa12345 authored on Mar 20			
Delete 0320	Verified	0006aa2	📄 📁 ↔
Akaaaa12345 authored on Mar 20			
Create 0320	Verified	a52cb8a	📄 📁 ↔
Akaaaa12345 authored on Mar 20			

1 2 3 4 변수, 수식, 그리고 입력(Input)

데이터를 입력받고, 저장하고, 계산하는 프로그램의 가장 기본적인 흐름을 정리한 공간입니다.

1. 데이터 입력 (Input): 사용자에게 값 받기

- `input()` 함수: 사용자로부터 키보드로 값을 입력받는 함수
- 주의점: `input()` 으로 입력받은 모든 값은 기본적으로 `**문자열(str)**`로 인식됨
- 형변환: 숫자로 계산하려면 `int(input())` (정수)이나 `float(input())` (실수)으로 감싸야 함

2. 변수 (Variable): 데이터 저장 공간

- 개념: 값을 담는 상자 (`변수명 = 값` 형태로 대입)
- 특징: 자료형(정수, 실수, 문자열 등)을 파이썬이 자동으로 판단
- 명명 규칙: 영문/숫자/_ 활용 (단, 숫자로 시작 불가, 예약어 사용 불가)

3. 수식과 연산자 (Expression & Operator)

- 산술 연산자: `+` (더하기), `-` (빼기), `*` (곱하기), `/` (나누기)
- 특수 연산자: `//` (몫), `%` (나머지), `**` (거듭제곱)
- 우선순위: 괄호 `()` 내부 > 거듭제곱 > 곱셈/나눗셈 > 덧셈/뺄셈

3주차

조건문

Commits on Mar 27, 2026		
Add files via upload	Verified	128c47d
Akaaaa12345 authored on Mar 27		
Commits on Mar 20, 2026		
Add files via upload	Verified	be1ec46
Akaaaa12345 authored on Mar 20		
Add files via upload	Verified	437a450
Akaaaa12345 authored on Mar 20		
Add files via upload	Verified	7df9148
Akaaaa12345 authored on Mar 20		
Create readme.md	Verified	8fd0130
Akaaaa12345 authored on Mar 20		
Delete 0320	Verified	ea2eebd
Akaaaa12345 authored on Mar 20		
readme.md	Verified	c02d4c4
Akaaaa12345 authored on Mar 20		
Delete 0320	Verified	0006aa2
Akaaaa12345 authored on Mar 20		
Create 0320	Verified	a52cb8a
Akaaaa12345 authored on Mar 20		
Delete 0320	Verified	00eda12
Akaaaa12345 authored on Mar 20		
readme.md	Verified	6a203ca
Akaaaa12345 authored on Mar 20		

조건문 (Conditional Statement)

주어진 조건에 따라 프로그램의 실행 흐름을 제어하는 조건문의 핵심 개념을 정리한 공간입니다.

1. 조건문(if)의 기본 개념

- 개념: "만약 조건이 참(True)이면 코드를 실행하고, 거짓(False)이면 건너뛰다!"
- 들여쓰기(Indentation): 파이썬은 조건문 아래 실행할 코드를 반드시 **들여쓰기(공백 4칸 또는 Tab)**로 구분함 (안 하면 에러 발생)

[Image of if else statement flowchart]

2. 조건문의 3가지 구조

- if 문: 조건이 참일 때만 특정 코드를 실행
- if - else 문: 조건이 참일 때와 거짓일 때 실행할 코드를 각각 나누어 처리
- if - elif - else 문: 여러 개의 조건을 차례대로 검사하여 해당하는 블록만 실행 (다중 조건 처리)

3. 비교 연산자와 논리 연산자

- 비교 연산자: == (같다), != (다르다), > , < , >= , <=
- 논리 연산자:
 - and : 두 조건이 모두 참이어야 참
 - or : 두 조건 중 하나만 참이어야 참
 - not : 참을 거짓으로, 거짓을 참으로 뒤집음

4주차

반복문

Commits on Apr 3, 2026

Add files via upload

Akaaaa12345

 authored on Apr 3

Verified

6a4fdc5

<>

반복문 (Loop)

정해진 조건이나 횟수에 따라 코드를 반복 실행하여 프로그램의 효율을 극대화하는 반복문의 핵심 개념을 정리한 공간입니다.

1. 반복문의 두 가지 종류

- `for` 문 (횟수 제어 반복): 대상을 지정하거나 정해진 횟수만큼 깔끔하게 반복할 때 사용
- `while` 문 (조건 제어 반복): 특정 조건이 참(True)인 동안 끝없이 반복할 때 사용 (조건이 거짓이 되면 탈출)

2. 반복문의 핵심 파트너

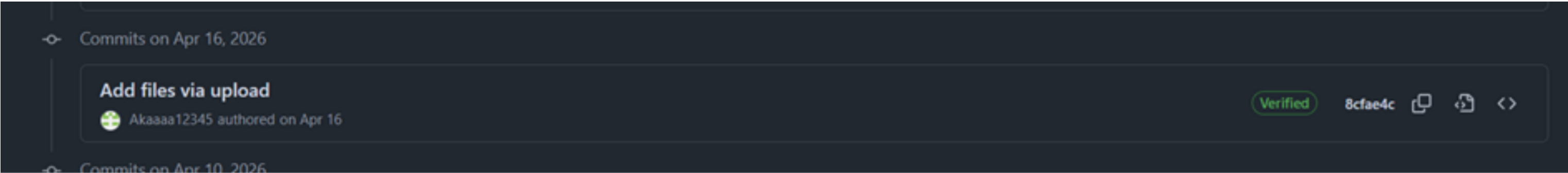
- `range()` 함수: `for` 문과 함께 쓰이며 연속된 숫자들을 생성해 줌
 - `range(5)` : 0부터 4까지 (총 5번)
 - `range(1, 10, 2)` : 1부터 9까지 2씩 건너뛰며 생성
- 무한 루프(Infinite Loop): `while` 문의 조건이 계속 참일 때 발생하며, 프로그램을 종료하려면 `Ctrl + C` 를 눌러야 함

3. 반복문의 흐름 제어 (Break & Continue)

- `break` : 반복문이 실행 중이더라도 즉시 그 자리에서 **반복문을 완전히 탈출**함
- `continue` : 남은 코드를 실행하지 않고, ****다음 반복 순서로 바로 패스(건너뛰기)****함

5주차

함수



📌 함수 (Function)

특정 기능을 수행하는 코드들을 하나로 묶어 재사용 가능하게 만드는 함수의 핵심 개념을 정리한 공간입니다.

1. 함수의 기본 개념 및 장점

- 개념: 프로그램 안의 작은 프로그램 (특정 입력값을 받아 처리하고 결과값을 돌려주는 상자)
- 장점: * 중복 코드를 제거하여 코드가 깔끔해짐 (코드의 재사용성)
 - 에러가 났을 때 해당 함수만 고치면 되므로 유지보수가 편해짐

2. 함수의 구조 (정의와 호출)

- 함수 정의 (`def`): `def` 함수이름 (매개변수): 형태로 새로운 함수를 만들
- 함수 호출: 정의한 함수 이름과 필요한 값을 넣어 실제로 실행시킴
- 인수(Argument) vs 매개변수(Parameter):
 - 매개변수: 함수를 정의할 때 값을 받는 변수 (함수 내부의 변수)
 - 인수: 함수를 호출할 때 함수에 실제로 전달하는 값

3. 반환값(Return)과 변수의 범위

- `return` 문: 함수가 실행을 마치고 자신을 호출한 곳으로 결과값을 돌려줄 때 사용 (값을 반환하고 함수는 즉시 종료됨)
- 지역 변수 (Local Variable): 함수 내부에서 만들어져 함수 안에서만 사용할 수 있는 변수
- 전역 변수 (Global Variable): 함수 외부(프로그램 전체)에서 만들어져 어디서나 접근할 수 있는 변수

6주차

파이썬 구조자료 리스트

Commits

History for `Python` / 0417 on `main`

All users

All time

Commits on Jun 11, 2026

Update readme.md

Akaaaa12345

 authored 51 minutes ago

Verified

 fe25cec

Commits on Apr 24, 2026

Add files via upload

Akaaaa12345

 authored on Apr 24

Verified

 360f915

Create readme.md

Akaaaa12345

 authored on Apr 24

Verified

 451e3fa

End of commit history for this file

📄 리스트 자료구조 (List)

여러 개의 데이터를 하나의 변수에 순서대로 모아서 관리하는 파이썬의 가장 대표적인 자료구조를 정리한 공간입니다.

1. 리스트의 기본 개념

- **개념:** 대괄호(`[]`)를 사용하며, 여러 가지 형태의 데이터(숫자, 문자열 등)를 콤마(`,`)로 구분하여 하나의 상자에 담는 것
- **인덱싱 (Indexing):** 리스트 안의 방 번호는 0번부터 시작함 (뒤에서부터 셀 때는 `-1` 번부터 시작)
- **슬라이싱 (Slicing):** `list[시작:끝]` 형태로 리스트의 일부분을 잘라서 가져옴 (끝 번호 바로 직전까지만 가져옴)

2. 리스트 주요 함수 및 연산

- **추가:** `.append(값)` (맨 뒤에 추가), `.insert(인덱스, 값)` (원하는 위치에 삽입)
- **삭제:** `del 리스트[인덱스]` (해당 방 데이터 삭제), `.remove(값)` (특정 값을 찾아서 삭제)
- **정렬:** `.sort()` (오름차순 정렬), `.reverse()` (순서 뒤집기)
- **기타:** `len(리스트)` (리스트의 전체 요소 개수 반환)

파이썬 구조자료 (튜플 , 딕셔너리 , 세트 , 문자열)

Commits

History for `Python / 0508` on `main`

All usersAll time

Commits on Jun 11, 2026

Update readme.md

Akaaaa12345 authored 52 minutes ago

Verified84a0817

Commits on May 15, 2026

Add files via upload

Akaaaa12345 authored last month

Verifieddcc9947

Commits on May 14, 2026

Create readme.md

Akaaaa12345 authored last month

Verifiedbf4d869

End of commit history for this file

📁 자료구조 (Tuple, Dictionary, Set) & 문자열 (String)

파이썬에서 제공하는 다양한 자료구조의 특징과 문자열을 효율적으로 다루는 방법을 정리한 공간입니다.

1. 튜플 (Tuple): 수정할 수 없는 리스트

- 특징: 소괄호 (`()`)를 사용하며, 리스트와 거의 같지만 한 번 생성하면 값을 변경하거나 삭제할 수 없음
- 용도: 프로그램이 실행되는 동안 절대로 바뀌지 않아야 하는 고정된 데이터(예: 캘린더의 요일, 좌표 등)를 안전하게 보관할 때 사용

2. 딕셔너리 (Dictionary): 사전형 자료구조

- 특징: 중괄호 (`{}`)를 사용하며, **`** Key(키): Value(값) **`**의 쌍으로 데이터를 관리함
- 용도: 데이터의 순서(인덱스) 대신 의미 있는 이름(Key)으로 값을 찾고 싶을 때 사용 (단, Key 값은 중복될 수 없음)
- 예시: `phone_book = {"홍길동": "010-1234-5678"}`

3. 세트 (Set): 집합 자료구조

- 특징: 중괄호 (`{}`)를 사용하며, 수학의 집합처럼 중복을 허용하지 않고 순서가 없음
- 용도: 리스트 등에서 중복된 데이터를 싹 제거하거나, 교집합(`&`), 합집합(`|`), 차집합(`-`) 같은 집합 연산이 필요할 때 활용

4. 문자열 (String)

- 특징: 문자들의 나열로, 리스트처럼 인덱싱과 슬라이싱을 통해 원하는 글자만 싹 골라낼 수 있음 (튜플처럼 원본 수정은 불가능)
- 주요 함수:
 - `* .upper() / .lower() :` 대소문자 변환
 - `.split() :` 특정 기준(공백 등)으로 문자열을 쪼개서 리스트로 만들
 - `.replace(A, B) :` 문자열 안의 A를 B로 변경

8주차

객체와 클래스

Commits

History for `Python` / 0515 on `main`

All usersAll time

Commits on Jun 11, 2026

Update readme.md

Akaaaa12345 authored 52 minutes ago

Verified7756ebe

Commits on May 21, 2026

Add files via upload

Akaaaa12345 authored 3 weeks ago

Verified12e216a

Create readme.md

Akaaaa12345 authored 3 weeks ago

Verifieda936c50

End of commit history for this file

객체(Object)와 클래스(Class)

코드를 독립적인 단위로 나누어 관리하는 **객체 지향 프로그래밍(OOP)**의 핵심 개념을 정리한 공간입니다.

1. 클래스와 객체의 개념 (붕어빵 틀과 붕어빵)

- 클래스 (Class):** 객체를 만들기 위한 **설계도** 또는 **'붕어빵 틀'**입니다. 데이터(속성)와 기능(메서드)을 정의한 **해둔 상태**입니다.
- 객체 (Object) / 인스턴스 (Instance):** 설계도(클래스)를 바탕으로 **실제 메모리에 만들어진 '결과물'** 또는 **'붕어빵'**입니다. 하나의 클래스로 여러 개의 서로 다른 객체를 찍어낼 수 있습니다.

2. 클래스의 구성 요소: 속성과 메서드

- 속성 (Attribute):** 객체가 가지고 있는 **'상태'**나 **'데이터'**를 말하며, 클래스 안의 **변수로** 표현됩니다. (예: 자동차의 색상, 현재 속도)
- 메서드 (Method):** 객체가 할 수 있는 **'행동'**이나 **'기능'**을 말하며, 클래스 안의 **함수로** 표현됩니다. (예: 가속하기, 브레이크 밟기)

3. 파이썬 클래스 핵심 문법

- `class` 키워드: 새로운 클래스를 정의할 때 사용합니다.
- 생성자 (`__init__`): 객체가 생성될 때 **자동으로 호출되는 특별한 메서드**로, 객체의 속성(초기값)을 설정할 때 사용합니다.
- `self` 키워드: 생성되거나 실행 중인 **객체 자신**을 가리키는 파이썬만의 규칙입니다. 메서드를 정의할 때 첫 번째 매개변수로 반드시 적어주어야 합니다.

2026 - 6- 12

단위 변환 프로그램

단위 변환기

센티미터(cm)를 입력하세요

변환하기

결과가 여기에 표시됩니다

단위 변환기

센티미터(cm)를 입력하세요

변환하기

결과: 4.72 인치

```
import tkinter as tk

# 1. 계산 기능 함수 (버튼 누르면 실행됨)
def convert():
    cm = float(entry.get())    # 입력창에서 글자 가져와서 숫자로 변환
    inch = cm / 2.54          # 인치 계산
    result.config(text=f"결과: {inch:.2f} 인치") # 결과 글자 바꾸기

# 2. 메인 윈도우 창 만들기
window = tk.Tk()
window.title("단위 변환기")
window.geometry("300x150") # 창 크기

# 3. 화면에 들어갈 글자, 입력창, 버튼 배치
label = tk.Label(window, text="센티미터(cm)를 입력하세요")
label.pack(pady=5)

entry = tk.Entry(window) # 입력받는 칸
entry.pack(pady=5)

# 버튼을 누르면 위에 만든 convert 함수가 실행됨
button = tk.Button(window, text="변환하기", command=convert)
button.pack(pady=5)

result = tk.Label(window, text="결과가 여기에 표시됩니다")
result.pack(pady=5)

# 4. 창 계속 띄워두기
window.mainloop()
```

느낀점

"이번 프로그래밍 입문 수업을 통해 파이썬을 처음 배웠습니다. 학기 초반에 배우던 (print), 변수, 조건문까지는 직관적이라 할 만하다고 생각했는데, 후반으로 갈수록 개념이 심화되면서 생각보다 어렵게 느껴졌습니다.

그러나 이번에 직접 가벼운 프로그램을 만들어보는 프로젝트를 진행하면서, 그동안 배웠던 내용들을 하나씩 되돌아보고 점검할 수 있었습니다. 특히 책으로만 볼 때는 헛갈렸던 문법들을 코딩 과정에서 직접 머릿속에 다시 한번 되새기는 좋은 시간이 되었습니다. 이번 경험을 발판 삼아 다음 학기에는 스스로 더 도전적이고 어려운 주제를 선택해 코딩해 보고 싶습니다."

점수

깃허브 점수 15/15

프로젝트 점수 15/15

총 30/30

Thank you